

Resolva os seguintes exercícios:

1. Nas instruções abaixo é alocado um vetor de inteiros com os valores [4, 3, 2, 1] e declarada uma variável sum com valor inicial igual a 4 (valor da primeira posição do vetor).
Sem utilizar um ciclo, acrescente instruções por forma a que a variável sum acumule os restantes valores do vetor, ficando com a sequência de valores: 4, 7, 9, 10.

```
let numbers = [4, 3, 2, 1];  
let sum = numbers[0];
```

2. Defina uma função que calcula o somatório dos valores de um vetor de inteiros.
3. As instruções em baixo criam um vetor de inteiros com comprimento 10. Escreva instruções para preencher o mesmo com os dígitos de 0 a 9, utilizando um ciclo while.

```
let digits = [];  
digits [9]= undefined;
```

4. Defina uma função por forma a que seja devolvido um vetor com os números naturais até um n (parâmetro).
5. Defina uma função isOrdered para verificar se um vetor numérico está ordenado por ordem crescente.
6. Defina uma função inverted para criar um vetor com os valores de outro vetor invertidos.

Exemplos:

```
inverted([1, 2, 3, 4]) → [4, 3, 2, 1]
```

7. Defina uma classe com funções auxiliares para consultar vetores de inteiros:

isEmpty: devolve verdadeiro se o vetor for vazio (comprimento zero);

first: devolve o primeiro valor do vetor, verificando que o vetor não é vazio;

last: devolve o último valor do vetor, verificando que o vetor não é vazio;

nextIndex: dado um vetor e um índice (válido), devolve o próximo índice (fazendo uma "rotação" do último para o primeiro).

prevIndex: dado um vetor e um índice (válido), devolve o índice anterior (fazendo uma "rotação" do primeiro para o último).

`element`: devolve o valor no índice dado no intervalo `[-comprimento, comprimento[`, onde os índices negativos acedem ao vetor a contar do fim.

Exemplos:

`isEmpty([]) → true`

`isEmpty([10, 15]) → false`

`first([9, 8, 7]) → 9`

`last([9, 8, 7]) → 7`

`nextIndex([1, 2, 3, 4], 2) → 3`

`nextIndex([1, 2, 3, 4], 3) → 0`

`prevIndex([1, 2, 3, 4], 0) → 3`

`element([2, 3, 4, 5], 2) → 4`

`element([2, 3, 4, 5], -1) → 5`

8. Defina uma classe com duas funções para copiar (replicar) vetores:

`copyNewSize`: replica um vetor tendo em conta um novo comprimento, por ordem crescente;

`copy`: replica um vetor (implementar utilizando `copyNewSize`).

Exemplos:

`copyNewSize([1, 3, 4], 5) → [1, 3, 4, 0, 0]`

`copyNewSize([1, 3, 4, 5], 2) → [1, 3]`

`copyNewSize([1, 2, 3], 3) → [1, 2, 3]`

`copy([1, 2, 3]) → [1, 2, 3]`

9. Defina uma classe com três funções relacionadas com aleatoriedade em vetores.

`randomArray`: para construir um vetor com dígitos aleatórios `[0, 9]`, dado um comprimento;

`randomIndex`: para sortear um índice válido para um vetor;

`randomElement`: para sortear um valor contido num vetor.

Exemplos:

`randomArray(5) → [3, 1, 0, 9, 5]`

`randomIndex([5, 6, 7, 8]) → 2`

`randomElement([5, 6, 7, 8]) → 8`

10. Escreva uma função para verificar se um número está contido num vetor.

Exemplos:

`contains([1, 3, 4, 2], 5) → false`

`contains([1, 3, 2, 1], 3) → true`

11. Escreva uma função para contar o número de ocorrências de um elemento num vetor.

Exemplos:

count([2, 3, 4, 2, 2], 2) → 3

count([1, 3, 2, 1], 7) → 0

12. Escreva uma função para verificar se um vetor de caracteres é palíndromo (a leitura é igual nos dois sentidos).

Exemplos:

isPalindrome(['s','o','p','a','p','o','s']) → true

isPalindrome(['s','o','p','a']) → false

13. Escreva um módulo com funções auxiliares para obter informações sobre os valores contidos num vetor numérico:

min: devolve o valor mínimo;

max: devolve o valor máximo;

sum: devolve o somatório;

average: devolve a média (utilizando sum).

Exemplos:

min([4.2, 2.1, 3.3, 5.0]) → 2.1

max([4.2, 2.1, 3.3, 5.0]) → 5.0

sum([4.2, 2.1, 3.3, 5.0]) → 14.6

average([4.2, 2.1, 3.3, 5.0]) → 3.65

14. Escreva uma função para verificar se dois vetores são iguais.
15. Escreva uma função que dados dois vetores (left e right), devolve um novo que resulta da junção de left e right.

Exemplos:

merge([1, 2], [3, 4, 5]) → [1, 2, 3, 4, 5]

16. Escreva três funções para obter sub-vetores de um vetor.

subArray: dados um índice de início e um de fim, é devolvido o sub-vetor nesse intervalo de índices;

leftSide: é devolvido o sub-vetor da metade esquerda; (invocar subArray)

rightSide: é devolvido o sub-vetor da metade esquerda; (invocar subArray)

(leftSide e rightSide têm um parâmetro booleano para incluir o elemento do meio, caso o comprimento do vetor seja ímpar)

Exemplos:

subArray([5, 6, 7, 8, 9], 1, 3) → [6, 7, 8]

leftSide([5, 6, 7, 8], false) → [5, 6]

rightSide([5, 6, 7, 8, 9], true) → [7, 8, 9]

17. Escreva uma função para verificar se um vetor de booleanos tem valores alternados.

Exemplos:

alternatedBooleans([true, false, true, false]) → true

alternatedBooleans([true, true, true, false]) → false

18. Escreva uma função para obter um vetor de booleanos invertendo os valores de outro vetor de booleanos.

Exemplos:

invertedBooleans([true, false, true]) → [false, true, false]